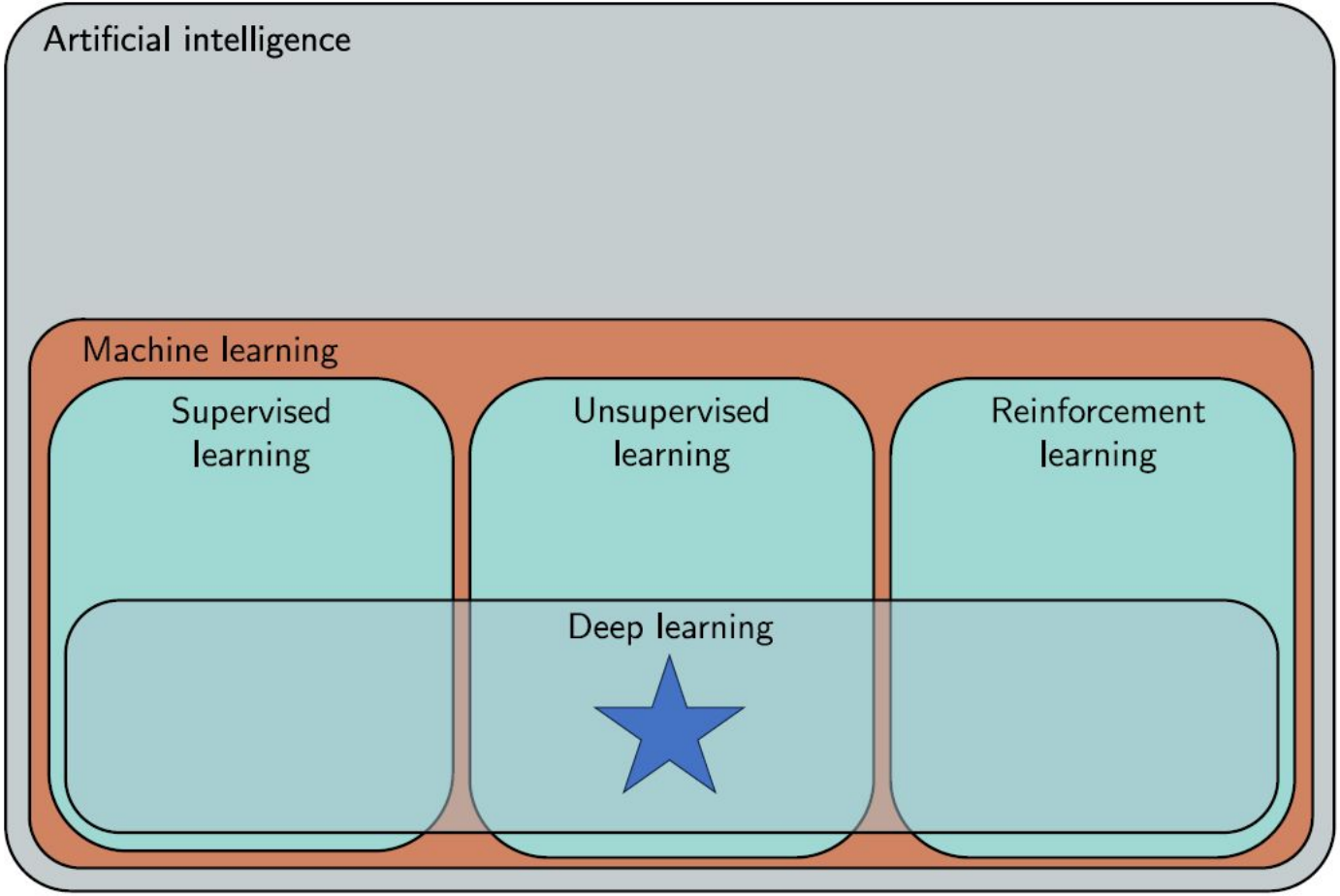
Large Language Models Fundamentals

Emrah Inan

Large Language Models for Research

- 1958 Perceptron (Simple `neural' model)
- 1986 Backpropagation (Practical Deep Neural networks)
- 1989 Convolutional networks (Supervised learning)
- 2012 AlexNet Image classification (Supervised learning)
- 2014 Generative adversarial networks (Unsupervised learning)
- 2014 Deep Q-Learning -- Atari games (Reinforcement learning)
- 2016 AlphaGo (Reinforcement learning)
- 2017 Machine translation (Supervised learning)
- 2019 Language models ((Un)supervised learning)
- 2022 Dall-E2 Image synthesis from text prompts ((Un)supervised learning)
- 2022 ChatGPT ((Un)supervised learning)
- 2023 GPT4 Multimodal model ((Un)supervised learning)

Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/



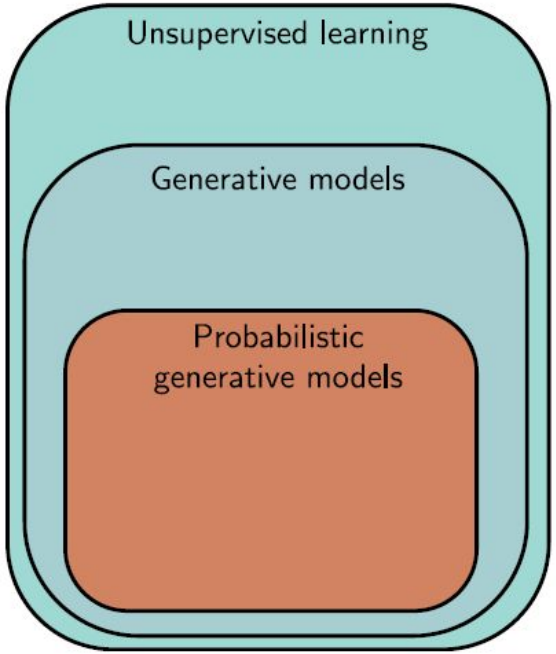
Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Unsupervised Learning

- Learning about a dataset without labels
  e.g., clustering

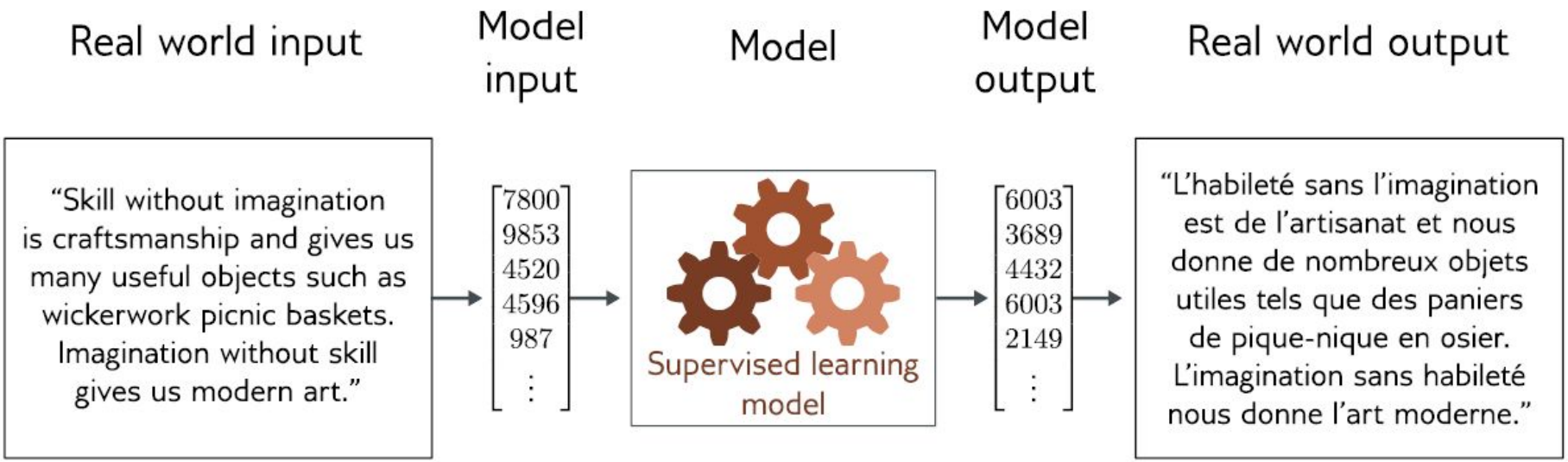
- Generative models can create examples
  e.g., generative adversarial networks

- PGMs learn distribution over data
  e.g., variational autoencoders,
  e.g., normalizing flows,
  e.g., diffusion models



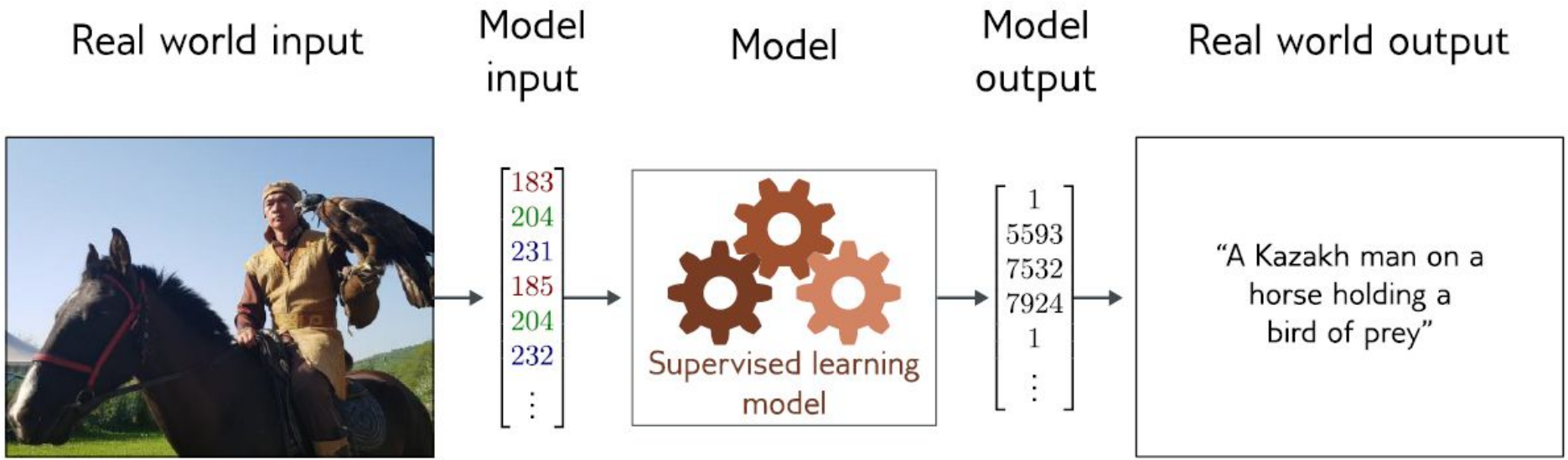
Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Translation



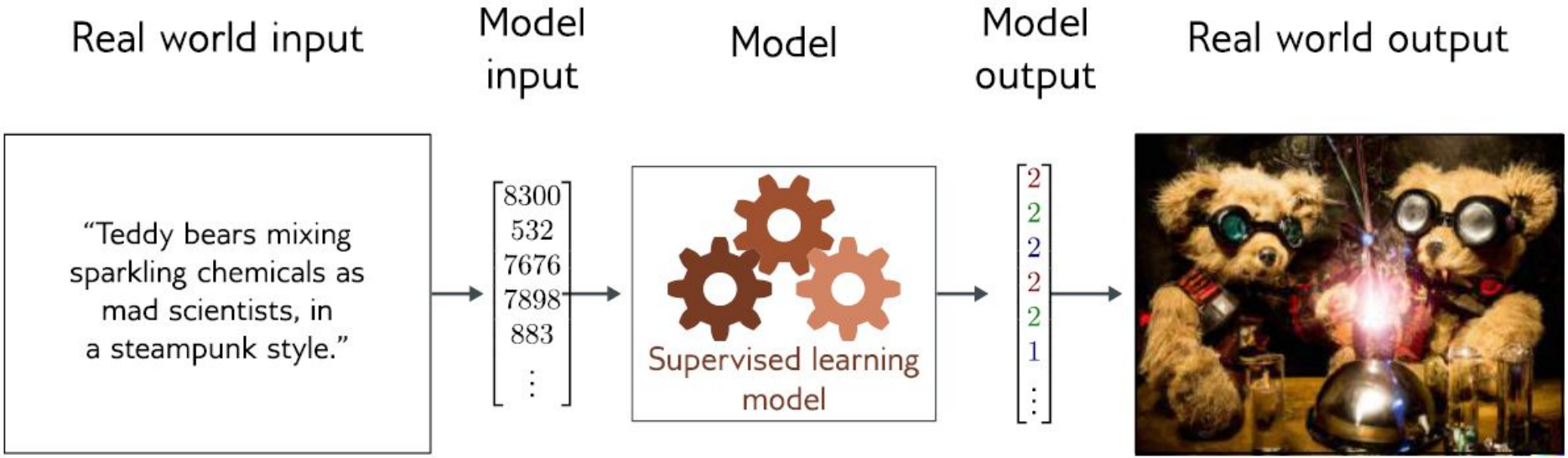
Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Image captioning



Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Image generation from text



Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

What do these examples have in common?

- Very complex relationship between input and output
- Sometimes may be many possible valid answers
- But outputs (and sometimes inputs) obey rules

"A Kazakh man on a horse holding a bird of prey"

Language obeys grammatical rules

Natural images also have "rules"

Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Idea

Learn the “grammar” of the data from unlabeled examples

Can use a gargantuan amount of data to do this (as unlabeled)

Make the supervised learning task earlier by having a lot of knowledge of possible outputs

Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

I was a little nervous before my first lecture at the University of Bath. It seemed like there were hundreds of students and they looked intimidating. I stepped up to the lectern and was about to speak, when something bizarre happened.

Suddenly, the room was filled with a deafening noise, like a giant roar. It was so loud that I couldn't hear anything else and I had to cover my ears. I could see the students looking around, confused and frightened. Then, as quickly as it had started, the noise stopped and the room was silent again.

I stood there for a few moments, trying to make sense of what had just happened. Then I realized that the students were all staring at me, waiting for me to say something. I tried to think of something witty or clever to say, but my mind was blank. So I just said, "Well, that was strange," and then I started my lecture.

I was a little nervous before my first lecture at the University of Bath. It seemed like there were hundreds of students and they looked intimidating. I stepped up to the lectern and was about to speak, when something bizarre happened.

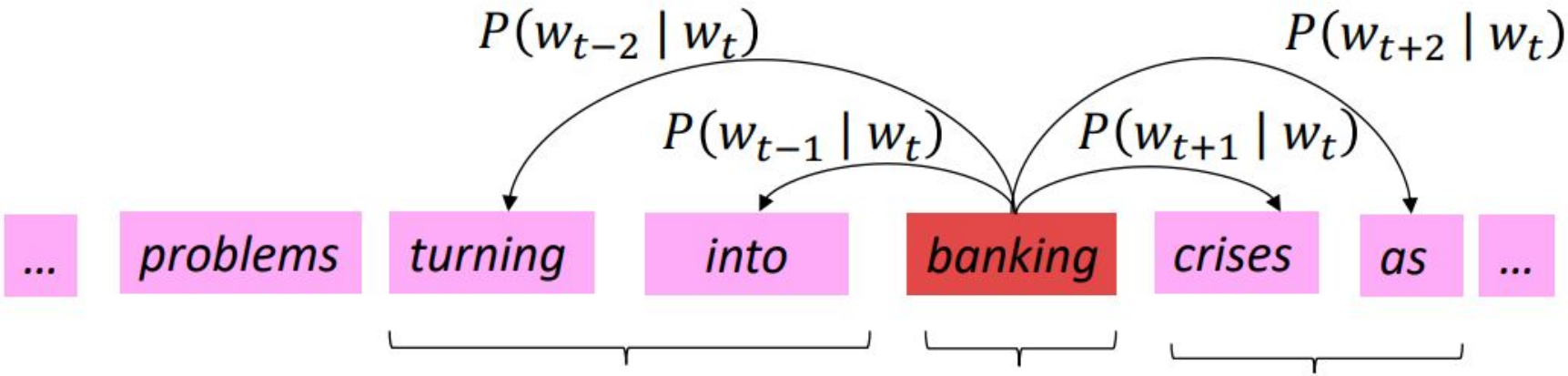
Suddenly, a giant rabbit ran into the lecture hall! The students started screaming and running around in panic. I was so shocked that I couldn't move. The rabbit ran up to me and hopped onto the lectern. Then, in a booming voice, it said:

"I am the Easter Bunny! I have come to give you all a special gift!"

The students were so surprised that they stopped screaming and listened to the Easter Bunny. Then, the Easter Bunny started handing out chocolate eggs to everyone in the lecture hall. The students were so happy that they started cheering and clapping. I was so relieved that the Easter Bunny had saved my lecture! After that, I was able to continue and the students paid attention for the rest of the hour. It was a great success!

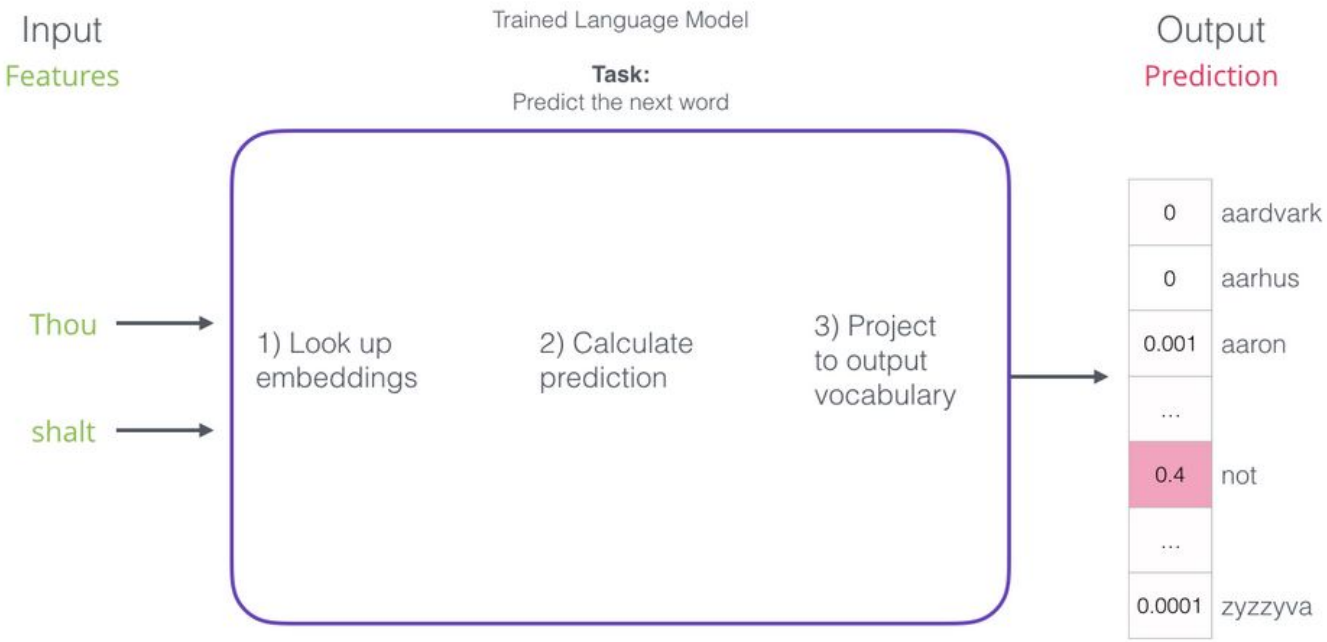
Simon J.D. Prince, Understanding Deep Learning, https://udlbook.github.io/udlbook/

Review: Main Idea of word2vec



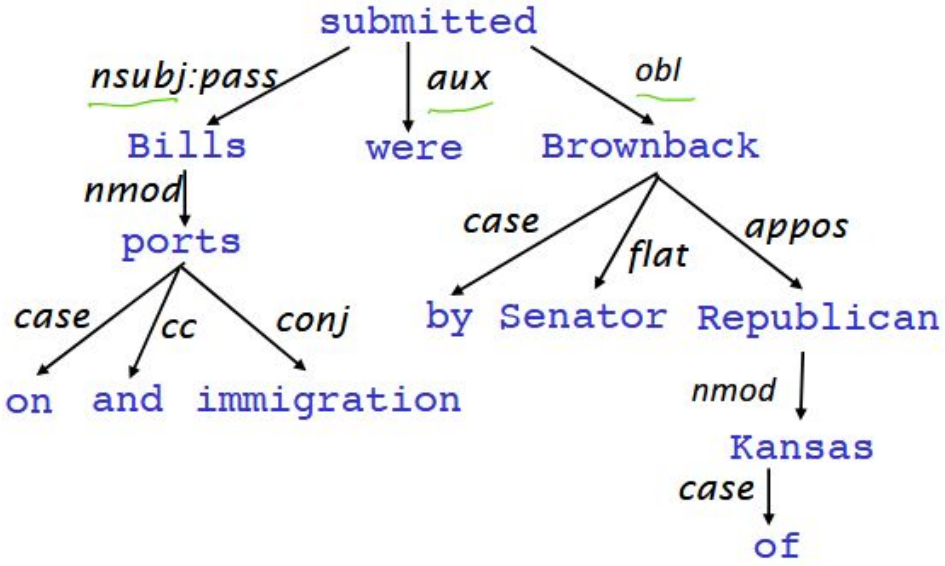
Christopher Manning, cs224n,
https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1234/readings/cs224n-2019-notes02-wordvecs2.pdf

Language Model: Next-word prediction



https://jalammar.github.io/illustrated-word2vec/

Review: Dependency Grammar and Dependency Structure



spaCy

Stanza

Christopher Manning, cs224n,
https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/slides/cs224n-2021-lecture04-dep-parsing-annotated.pdf

```python
from transformers import AutoModelForCausalLM, AutoTokenizer

# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=False,
)
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")
```

Although we can now use the model and tokenizer directly, it's much easier to wrap it in a `pipeline` object:

```python
from transformers import pipeline

# Create a pipeline
generator = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    return_full_text=False,
    max_new_tokens=500,
    do_sample=False
)
```

Finally, we create our prompt as a user and give it to the model:

```python
# The prompt (user input / query)
messages = [
    {"role": "user", "content": "Create a funny joke about chickens."}
]

# Generate output
output = generator(messages)
print(output[0]["generated_text"])
```

https://github.com/HandsOnLLM/Hands-On-Large-Language-Models/blob/main/chapter01/Chapter%201%20-%20Introduction%20to%20Language%20Models.ipynb

```python
def chatgpt_generation(prompt, document, model="gpt-3.5-turbo-0125"):
    """Generate an output based on a prompt and an input document."""
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant."
            },
        {
            "role": "user",
            "content":   prompt.replace("[DOCUMENT]", document)
            }
    ]
    chat_completion = client.chat.completions.create(
      messages=messages,
      model=model,
      temperature=0
    )
    return chat_completion.choices[0].message.content
```

```python
# Define a prompt template as a base
prompt = """Predict whether the following document is a positive or negative movie review:

[DOCUMENT]

If it is positive return 1 and if it is negative return 0. Do not give any other answers.
"""

# Predict the target using GPT
document = "unpretentious , charming , quirky , original"
chatgpt_generation(prompt, document)
```

```
'1'
```

https://github.com/HandsOnLLM/Hands-On-Large-Language-Models/blob/main/chapter04/Chapter%204%20-%20Text%20Classification.ipynb

Complex Prompt

```
# Text to summarize which we stole from https://jalammar.github.io/illustrated-transformer/ ;)
text = """In the previous post, we looked at Attention – a ubiquitous method in modern deep learning models. Attention is a
The Transformer was proposed in the paper Attention is All You Need. A TensorFlow implementation of it is available as a pa
Let's begin by looking at the model as a single black box. In a machine translation application, it would take a sentence i
Popping open that Optimus Prime goodness, we see an encoding component, a decoding component, and connections between them.
The encoding component is a stack of encoders (the paper stacks six of them on top of each other – there's nothing magical
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers:
The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the
The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is in
The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of
Now that we've seen the major components of the model, let's start to look at the various vectors/tensors and how they flow
As is the case in NLP applications in general, we begin by turning each input word into a vector using an embedding algorit
Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.
The embedding only happens in the bottom-most encoder. The abstraction that is common to all the encoders is that they rece
After embedding the words in our input sequence, each of them flows through each of the two layers of the encoder.
Here we begin to see one key property of the Transformer, which is that the word in each position flows through its own pat
Next, we'll switch up the example to a shorter sentence and we'll look at what happens in each sub-layer of the encoder.
Now We're Encoding!
As we've mentioned already, an encoder receives a list of vectors as input. It processes this list by passing these vectors
"""

# Prompt components
persona = "You are an expert in Large Language models. You excel at breaking down complex papers into digestible summaries.
instruction = "Summarize the key findings of the paper provided.\n"
context = "Your summary should extract the most crucial points that can help researchers quickly understand the most vital
data_format = "Create a bullet-point summary that outlines the method. Follow this up with a concise paragraph that encapsu
audience = "The summary is designed for busy researchers that quickly need to grasp the newest trends in Large Language Mod
tone = "The tone should be professional and clear.\n"
text = "MY TEXT TO SUMMARIZE"  # Replace with your own text to summarize
data = f"Text to summarize: {text}"

# The full prompt - remove and add pieces to view its impact on the generated output
query = persona + instruction + context + data_format + audience + tone + data
```

https://github.com/HandsOnLLM/Hands-On-Large-Language-Models/blob/main/chapter06/Chapter%206%20-%20Prompt%20Engineering.ipynb

In-Context Learning: Providing Examples

```
# Use a single example of using the made-up word in a sentence
one_shot_prompt = [
    {
        "role": "user",
        "content": "A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Giga
    },
    {
        "role": "assistant",
        "content": "I have a Gigamuru that my uncle gave me as a gift. I love to play it at home."
    },
    {
        "role": "user",
        "content": "To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:
    }
]
print(tokenizer.apply_chat_template(one_shot_prompt, tokenize=False))
```

```
<s><|user|>
A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru is:<|end|>
<|assistant|>
I have a Gigamuru that my uncle gave me as a gift. I love to play it at home.<|end|>
<|user|>
To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:<|end|>
<|assistant|>
```

```
# Generate the output
outputs = pipe(one_shot_prompt)
print(outputs[0]["generated_text"])
```

```
During the intense duel, the knight skillfully screeged his opponent's shield, forcing him to defend himself.
```

https://github.com/HandsOnLLM/Hands-On-Large-Language-Models/blob/main/chapter06/Chapter%206%20-%20Prompt%20Engineering.ipynb

Reasoning with Generative Models

Chain-of-Thought: Think Before Answering

```
# # Answering without explicit reasoning
# standard_prompt = [
#     {"role": "user", "content": "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis bal
#     {"role": "assistant", "content": "11"},
#     {"role": "user", "content": "The cafeteria had 25 apples. If they used 20 to make lunch and bought 6 more, how many a
# ]

# # Run generative model
# outputs = pipe(standard_prompt)
# print(outputs[0]["generated_text"])
```

```
The cafeteria started with 25 apples. They used 20 apples to make lunch, so they had:

25 - 20 = 5 apples left.

Then they bought 6 more apples, so they now have:

5 + 6 = 11 apples.
```

https://github.com/HandsOnLLM/Hands-On-Large-Language-Models/blob/main/chapter06/Chapter%206%20-%20Prompt%20Engineering.ipynb

Ollama: Quickstart

Download Ollama

Run a model

CLI Python JavaScript cURL

Open a terminal and run the command:

```
ollama run gemma3
```

https://docs.ollama.com/quickstart

Ollama: Models

Model	Parameters	Size	Download
Gemma 3	1B	815MB	`ollama run gemma3:1b`
Gemma 3	4B	3.3GB	`ollama run gemma3`
Gemma 3	12B	8.1GB	`ollama run gemma3:12b`
Gemma 3	27B	17GB	`ollama run gemma3:27b`
QwQ	32B	20GB	`ollama run qwq`
DeepSeek-R1	7B	4.7GB	`ollama run deepseek-r1`
DeepSeek-R1	671B	404GB	`ollama run deepseek-r1:671b`
Llama 4	109B	67GB	`ollama run llama4:scout`
Llama 4	400B	245GB	`ollama run llama4:maverick`
Llama 3.3	70B	43GB	`ollama run llama3.3`
Llama 3.2	3B	2.0GB	`ollama run llama3.2`

https://github.com/ollama/ollama

Ollama Python Library

Install

```
pip install ollama
```

Usage

```
from ollama import chat
from ollama import ChatResponse

response: ChatResponse = chat(model='gemma3', messages=[
  {
    'role': 'user',
    'content': 'Why is the sky blue?',
  },
])
print(response['message']['content'])
# or access fields directly from the response object
print(response.message.content)
```

https://github.com/ollama/ollama-python

```python
from ollama import chat

messages = [
  {
    'role': 'user',
    'content': 'Why is the sky blue?',
  },
  {
    'role': 'assistant',
    'content': "The sky is blue because of the way the Earth's atmosphere scatters sunlight.",
  },
  {
    'role': 'user',
    'content': 'What is the weather in Tokyo?',
  },
  {
    'role': 'assistant',
    'content': 'The weather in Tokyo is typically warm and humid during the summer months, with
  },
]

while True:
  user_input = input('Chat with history: ')
  response = chat(
    'gemma3',
    messages=[*messages, {'role': 'user', 'content': user_input}],
  )

  # Add the response to the messages to maintain the history
  messages += [
    {'role': 'user', 'content': user_input},
    {'role': 'assistant', 'content': response.message.content},
  ]
  print(response.message.content + '\n')
```

https://github.com/ollama/ollama-python/blob/main/examples/chat-with-history.py

THANKS

QUESTIONS?